

CSSE 220 Syllabus

Welcome and Overview

Welcome to Object-Oriented Software Development! We are Ian and Aaron (a.k.a. Profs. Ludden & Wilkin), and we'll be your instructors. Before we get into details, here's a quick overview. In CSSE 220, we will build on your foundational programming skills by exploring object-oriented design principles and other software engineering practices. We will use Java, a widely-used object-oriented programming language, to apply these concepts. If you continue on to other CSSE courses, this course will help you start thinking about the bigger picture of software development. If this is your last CSSE course, it will position you to write readable, maintainable code to support projects in your primary field.

Course Logistics

This is a 4-credit course requiring CSSE 120 as a prerequisite. See the [course catalog](#) for more details.

Time(s) and Location(s):

Both sections meet MWF in Olin 157 (O157). Section 01 is 8:00–9:50 a.m., and Section 04 is 3:00–4:50 p.m.

Course Description:

Object-oriented programming concepts, including the use of inheritance, interfaces, polymorphism, abstract data types, and encapsulation to enable software reuse and assist in software maintenance. Recursion, GUIs and event handling. Use of common object-based data structures, including stacks, queues, lists, trees, sets, maps, and hash tables. Space/time efficiency analysis. Testing. Introduction to UML.

Learning Outcomes:

Students who successfully complete this course should be able to:

1. Develop software that incorporates the following techniques:
 - (a) Inheritance and class hierarchies
 - (b) Interfaces
 - (c) Polymorphism
 - (d) Casting
 - (e) Exceptions
 - (f) Collections
 - (g) Event-driven graphical user interfaces
 - (h) Exploring and using large-scale API packages (e.g. GUI libraries)
 - (i) Recursion
2. Perform the following steps of the software development cycle effectively:
 - (a) Design expressed as UML class diagrams
 - (b) Documentation before coding
 - (c) Unit and system testing
3. Explain the implementation of linked lists
4. Analyze the asymptotic worst case run time of simple programs.
5. Predict the performance of simple algorithms, including search and sort, given their asymptotic worst, best, and average case run times.
6. Select basic data structures (i.e., arrays, linked lists) based on asymptotic time complexity of typical operations.
7. Work in a team of 2–4 students on a small-to-medium-size software development project including at least three iterative development cycles, demonstrating effective:
 - (a) Use of team roles
 - (b) Team decision making
 - (c) Division of labor
8. Identify opportunities and limitations of using AI tools in the software development cycle.

How to Get Help

We have lots of sources of help to facilitate your learning in CSSE 220. Don't resign yourself to struggling through this course alone. If you find that you have worked on something for 20 minutes without making any progress, it's probably time to seek help! Software development is a team sport, and the best developers know that a fresh set of eyes can often spot a problem right away.

Instructor Email and Drop-In Hours:

See [Ian's weekly schedule](#) (Section 01) or [Aaron's weekly schedule](#) (Section 04) for student drop-in hours or to book an appointment. **The most effective and efficient way to get help** is to come to drop-in hours, and it's one of our favorite parts of the job, so please stop by. If you cannot attend drop-in hours, please book an appointment (see link in schedule) or email us at luddenig@rose-hulman.edu or wilkin@rose-hulman.edu.

Learning Center Tutors:

The Learning Center in the Logan Library offers free peer tutoring for CSSE 220 and many other courses. See the Learning Center's [My Rose-Hulman page](#) for this quarter's tutoring schedule and more info. They also offer [Sophomore Resident Tutors \(SRTs\)](#) in the Percopo classroom in the evenings.

Homework Assignments

The course's homework assignments provide opportunities for deeper practice of object-oriented software development skills introduced in class. To get the greatest learning benefit, you should start assignments early, seek help when you get stuck, and review feedback on your submissions.

Unless an assignment's instructions explicitly allow/require the use of genAI tools or other resources, you should only use class resources and the Java API. Any time you get help while working on an assignment, whether from an instructor, classmate, TA, Learning Center tutor, or an online resource, you should cite the help near the top of your submitted file(s).

Homework assignments are for practice, so your final grade depends not on perfect accuracy in homework submissions but on overall completion thresholds, where *completion* requires making an honest effort and, if applicable, passing nearly all unit tests. See individual assignment instructions for more details.

Assignments will be submitted through Gradescope, which you can access by clicking the assignment links in Moodle. You can access your grades in Moodle and feedback in Gradescope. If you believe there has been an error, please contact your instructor.

Final Project

The course ends with a group project designed to integrate the concepts and skills learned throughout the course. This project will require collaboration, creativity, and critical thinking as you work in a team of 2–4 students to develop an object-oriented software application. Detailed guidelines and expectations for the final project will be provided in a separate document. Each project will earn one of four grades: Poor, Satisfactory, Good, or Excellent. Project application domains will vary, but every project must include the following features:

- File I/O with custom exception handling.
- A Java Swing GUI with event-based programming.
- Inheritance and interfaces with polymorphism.
- JUnit tests.
- Design analysis in terms of object-oriented design principles.
- Appropriate data structures, such as ArrayLists or HashMaps.
- A presentation during the last week of class.

Assessments and Final Grade

In place of the familiar three-midterm-exams-and-a-final assessment structure, this course uses a Mastery-Based Learning (MBL) approach. MBL is a [research-backed](#) way to encourage learning from failure and help you track and prioritize your learning progress.

List of Assessed Skills

The MBL assessment approach divides course learning targets into individual skills. (Each MBL skill is roughly equivalent to one “exam problem type” in a traditional structure.) Figure 1 shows how these skills are divided into three categories: Must-Have (MH), Important (IMP), and Advanced (ADV). Each category includes both theory/paper skills and coding skills. This list does not include the [Debugging Skill Check](#) (DSC), which will be assessed during drop-in hours or by appointment.

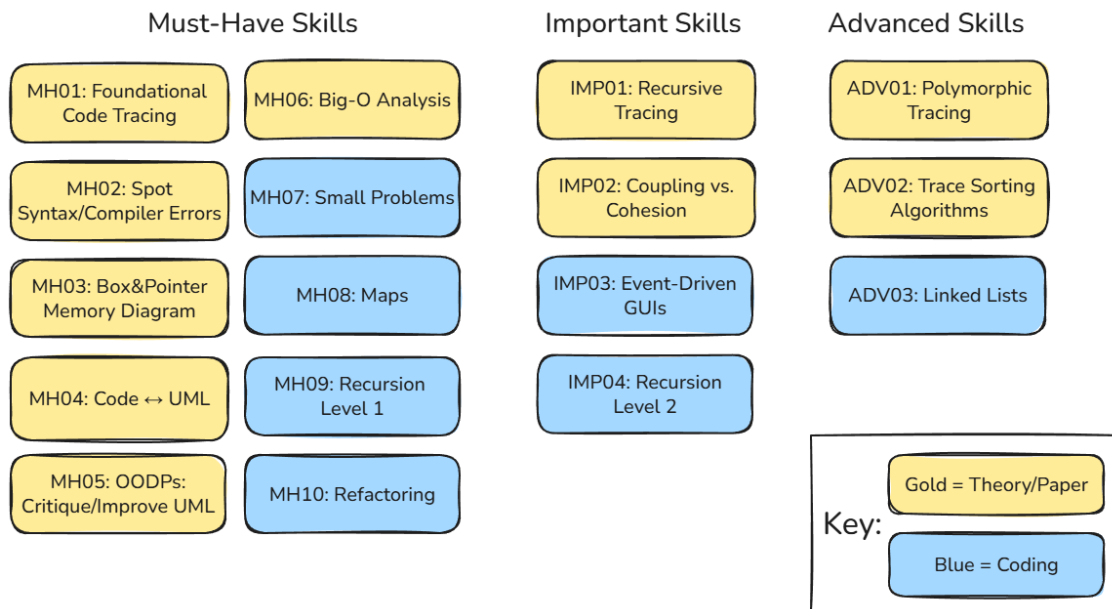


Figure 1: MBL Skills List for CSSE 220.

MBL Assessment Grading and Timing

Throughout the course, you will have multiple attempts to demonstrate each skill. Each attempt will be graded as “Met” or “Not Yet”. Once you earn a “Met” for a skill, you do not need to attempt that skill again (though you *will* use that skill in future assignments, especially the final project).

The theory/paper skills will be assessed during the second 50-minute class period, as indicated in the course calendar on Moodle. The coding skills will be assessed during the entire 110-minute class session, as indicated in the course calendar on Moodle. On each skill assessment day, you will select which of the available skills to attempt, which may include skills for which you received a “Not Yet” previously, by picking up the instructions sheets for those skills. See Figures 2 and 3 for the schedule of when each skill will be assessed for the first time.

The final exam slot will be used as a last opportunity to attempt skills you have not yet met. Only one new skill, ADV03, will be introduced during the final exam slot. There is no limit on the number of skills you may attempt during the final exam slot, but it is in your best interest to meet as many skills as possible *before* finals week.

Final Letter Grades

Table 1 shows the minimum requirements for each letter grade. (Failure to meet the minimum requirements for a D results in an F.) **We reserve the right to modify these requirements in a manner that makes some or all letter grades easier to attain.** We will not make any requirements harder.

Table 1: Requirements to earn each possible letter grade.

Final Grade	Minimum Requirements
D	• meet all but two MH skills • pass Debugging Skill Check • complete final project at “satisfactory” level or better • complete $\geq 60\%$ of homework assignments
D+	Meet all requirements for D, but also meet two more skills
C	• meet all MH skills • pass Debugging Skill Check • complete final project at “satisfactory” level or better • complete $\geq 70\%$ of homework assignments
C+	Meet all requirements for C, but also meet 1 IMP + 1 more IMP/ADV
B	• meet all MH skills • meet two IMP skills • pass Debugging Skill Check • complete final project at “good” level or better • complete $\geq 75\%$ of homework assignments
B+	Meet all requirements for B, but also meet two more IMP/ADV skills
A	• meet all MH and IMP skills • meet one ADV skill • pass Debugging Skill Check • complete final project at “good” level or better • complete $\geq 85\%$ of homework assignments

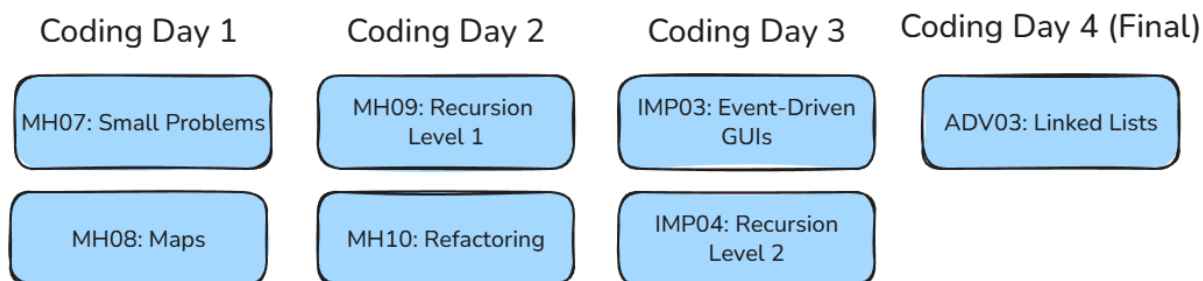


Figure 2: Schedule of Coding Skill Assessments

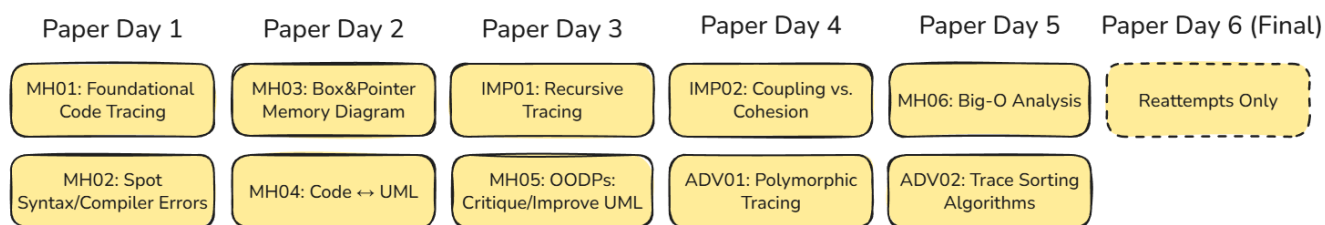


Figure 3: Schedule of Theory/Paper Skill Assessments

Additional Course Policies

Late Assignment/Incomplete Assignment Policy:

In general, you should plan ahead and submit assignments on time or early if you are able. Falling behind can cause many problems in this course. However, we understand that scheduling can be difficult at times. **You may request a 24-hour extension** (longer extensions are not generally considered due to the pace of the course) on an assignment using the **Extension Request Form** found just under the colorful Airtable Calendar at the top of the course Moodle site. If you do request an extension, you should do your best to request the extension before noon (12 p.m.) Eastern time of the day the assignment is due. There is no hard limit to how many extensions you may request, but we reserve the right to deny what appear to be unreasonable requests, or requests in which we believe granting the extension would cause more harm than good. Please remember that we expect students to act professionally. It can help to ask yourself whether you would make such a request to a manager while at work and whether the request is based on reasonable circumstances.

Here are some general guidelines:

- Send extension requests as soon as feasible, generally before noon (12:00 p.m.) on the date the assignment is due. If you are very ill, the timing is more flexible, but the sooner the better.
- It is OK to request an extension and then not use it.
- Putting the assignment off until the last moment (such as starting 4 hours before the deadline) is NOT a justification for an extension. We reserve the right to see your work-in-progress.
- **NOTE:** Most assignments will usually take more work than just sitting down and coding everything in an hour. You should plan accordingly and start your assignments early. The course Airtable calendar indicates when you should start looking at each assignment.
- The typical extension is 24 hours. On very rare occasions, we may allow 48 hours.
- We keep track of extensions per student. If you seem to be overusing the privilege, we may stop allowing extensions or make special arrangements for you.
- If the reason your work is late is because you are having trouble completing the assignment (e.g., a bug you can't fix or just difficulty understanding), reach out for help (see "How to Get Help"). Don't just assume that more time will fix the problem.
- Please understand that the schedule of assignments is very tight. Once you have completed one assignment, it is usually then time to start the next as soon as possible. So, if you request an extension on an assignment, please realize that extra time will now take time away from your next assignment. Don't let this snowball, causing every assignment to be later than the last.

Attendance Policy:

Attendance in class is an essential part of your learning. We trust that you will attend every class session unless an illness, athletic/extracurricular event, etc., prevents you from doing so. When you must miss class, please let me know as early as possible. After **four unexcused absences**, your final grade may

be reduced by **one full letter grade** for each additional unexcused absence. See the [Registrar's page](#) for additional information on institute-wide attendance policies.

Course Participation Policy:

When you are physically present in class, We expect you to also be *mentally* present and focused on CSSE 220 activities. If you and/or your group finishes an activity early, you can work on assignments, review/organize your notes, or take other actions that further your CSSE 220 learning. We know it may be tempting to hop over to an assignment for a different course, but context-switching hinders your learning (and [harms productivity of software developers](#)).

Course Academic Integrity Policy:

We believe that everyone in this course is fundamentally honest, and we will help you learn the conventions of academic integrity, such as citing sources when you use others' ideas. It is essential for all students to cite any and all sources of help received in completing coursework. This practice not only fosters a culture of honesty and transparency but also prevents misunderstandings that might otherwise escalate to formal proceedings. Students should also be aware of what is appropriate help on homework assignments—see [What Constitutes Misconduct](#). To ensure fairness and responsibility, any instances of suspected misconduct will be handled through the CSSE Integrity Committee.

If a case of suspected misconduct arises, it will be submitted to the CSSE Integrity Committee for review (see [Integrity Policy—Procedures](#) and possible penalties [IntegrityPilotPolicy—Penalties & Evidence](#)) Exceptions may be made for cases which the instructor deems minor or which arise late in the term. The instructor may choose to handle such cases directly (without CSSE Integrity Committee involvement). The process includes an initial review of the evidence by the committee, a time for students to explain or admit to potential misconduct, and potentially a hearing to examine the circumstances and evidence. Students are encouraged to continue their studies and engage with the course material and instructor normally throughout the investigation.

Generative AI:

It is expected that any work submitted for assessment represents the intellectual work of the individual(s) submitting the work. Any attempt to pass off the intellectual work of another (including the work generated by Large Language Models (LLMs) like ChatGPT) as their own or without proper attribution is an example of academic misconduct. **When assignments *do* allow/require the use of LLMs/genAI tools, clearly indicate which part(s) it generated, and include conversation links.**

Institute Policies

Students with Accessibility Needs:

Rose-Hulman is committed to working with students who have special needs or disabilities. Visit the [Accessibility Services website](#) for more information. Requests for academic accommodations must be documented with and approved by the Accessibility Services office before use in this course. **If you have difficulty accessing any course materials, please let your instructor know.**

Student Counseling:

The [Student Counseling Center](#) helps students identify barriers to success, build personal awareness, and develop life skills. Counseling appointments are free and confidential.

Academic Integrity:

The [Student Handbook](#) and Rose-Hulman's [Academic Rules and Procedures](#) describe penalties and processes invoked as a consequence if academic misconduct (such as cheating, plagiarizing, or interfering with the academic progress of other students) takes place. It is the responsibility of each student to know and follow Rose-Hulman's rules about academic integrity. See the Academic Integrity section in this syllabus for course-specific policies.

Community and Belonging:

As an institution devoted to excellence and innovation in education, we acknowledge our responsibility to create opportunities for growth and development, including the ability to interact with people of all backgrounds, experiences, and viewpoints. The [Office of Community and Belonging](#) seeks to advance a sense of belonging on campus where each individual is valued and accepted for their unique experiences, backgrounds, and aptitudes.

Online Access Requirements:

Rose-Hulman welcomes students from around the world, and encourages faculty, staff and students to travel around the world. However, geopolitical conditions and compliance with export law and regulations prevent us from delivering certain kinds of educational experiences and/or using certain kinds of Institute technologies in some locations. For example, students in locations with limited access to the internet in general, with restricted access to portions of the internet, or which are embargoed or sanctioned by the United States government may not be able to successfully complete Rose-Hulman courses.

Emergency Information:

To receive email or text messages regarding emergency situations that may impact campus and, possibly, the delivery of classes, [register for RAVE alerts](#) and/or follow [@RoseHulmanAlert](#). Any announcements about the Institute's ability to offer classes will be shared on Rose-Hulman's website.

Student Well-Being and Academic Success:

Rose-Hulman cares about the well-being of every student. Stress is a normal part of the college experience and is often compounded both by classwork and unexpected challenges outside the classroom. I encourage you to reflect on how you take care of yourself throughout the term, especially before the demands of exams and projects reach their peak. You can reach out to the [Learning Center](#), [Student Academic Success](#), or the [Student Counseling Center](#) (which offers both online and in-person resources) if you would like assistance. Additional resources to help maintain your work-life balance can be found on the [Campus Life](#) web page and within the [Student Success Guide](#). If you, or someone you know, is struggling, you can let us know through the [Rose Cares](#) system, and someone will reach out.

We reserve the right to modify the course content, schedule, topics, policies, etc. outlined in this syllabus. Any time the syllabus is updated, you will receive an email notification.